Candidate Transformer AI

AI-Powered Candidate Intelligence Platform · Spring Boot 3 · Java 21 · Apache PDFBox · Apache Commons CSV · Thymeleaf

Transforms a recruiter CSV and a candidate Resume PDF into a unified, AI-analysed candidate profile — in a single HTTP request.

Backend	Spring Boot 3 / Java 21
Architecture	Layered MVC
Input	CSV + Resume PDF
Output	Profile + JSON
AI	Rule-based Analyzer

PROJECT OVERVIEW

Recruiters spend significant time manually reviewing resumes and cross-referencing spreadsheets before shortlisting candidates.

This system accepts a structured Recruiter CSV and an unstructured Resume PDF, then automatically parses, normalises, merges both sources, and runs an AI analysis pipeline — producing a unified candidate record in under two seconds.

The output is a recruiter-ready profile page and a downloadable JSON report that a hiring manager can review without any technical context.

Business value: eliminates manual resume review, reduces screening time, and produces consistent, bias-free candidate summaries at scale.

SYSTEM ARCHITECTURE

Recruiter
Upload CSV + Resume PDF

↓

Transform Controller
POST /transform

↓

Transform Service
Orchestration layer

↓

CSV Parser · Resume Parser
Commons CSV · PDFBox

↓

Normalizer · Merger
Canonical data + field merge

↓

AI Analyzer
Role · Score · Gaps · Summary

↓

Candidate Profile · JSON Export
Thymeleaf · /download/json

END-TO-END WORKFLOW

1	**Upload**	CSV + Resume PDF via web form
2	**Parse**	CSV headers mapped; PDF split into sections
3	**Normalise**	Names, emails, phones, skills standardised
4	**Merge**	CSV metadata + resume content unified
5	**Analyse**	9 role clusters scored; match score computed
6	**Profile**	Recruiter-friendly report rendered in browser
7	**Export**	Clean JSON served as named download file

TECHNOLOGY STACK

Java 21	Spring Boot 3
Apache PDFBox	Commons CSV
Jackson	Thymeleaf
Bootstrap 5	Maven

KEY FEATURES

CSV Upload	Resume PDF Upload
Section-based Parsing	Data Normalisation
CSV + Resume Merge	AI Recruiter Insights
Candidate Profile UI	Recruiter JSON Export

KEY DESIGN DECISIONS

Section-based resume parsing
Splitting the PDF by section headings prevents content from one section — such as certifications or project descriptions — from being misclassified as technical skills.

Technical skill whitelist
Matching parsed tokens against a curated list of ~120 real technologies eliminates false positives like internship titles, certificate names, and random text.

Structured + unstructured data merge
Combining the recruiter's CSV (contact metadata) with the resume (skills, education, experience) provides a richer, unified candidate record than either source alone.

Rule-based recruiter insights
Scoring candidate skills across nine predefined role clusters runs entirely in-process — zero API cost, deterministic results, and works fully offline.

Recruiter-readable JSON export
The JSON report is assembled from parsed data into human-readable sections so a hiring manager can understand the candidate profile without technical knowledge.

EDGE CASES HANDLED

Fresher resume — yearsExperience = 0, labelled "Fresher"

Resume without Skills section — returns empty skill list

Certifications adjacent to Skills — stop condition blocks extraction

Missing email or phone — stored empty, no null crash in view

Duplicate skills — deduplicated and sorted alphabetically

Missing CSV fields — graceful empty-string fallback, no crash

FUTURE IMPROVEMENTS

→ Job Description Matching — rank candidates against a specific JD

→ LLM Candidate Summary — GPT-powered narrative instead of rule-based

→ Batch Resume Processing — multiple PDFs in a single session